DAY 01 | POST-READ

Python Basics: Recap and Beyond

What we covered today, plus a few ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Setup	Colab runs Python in the browser. VS Code runs it on your laptop. Both run the same code.
print()	Shows values on screen. Separate several values with commas.
Variables	A name pointing to a value. = stores, it does not compare.
Data types	int (whole), float (decimal), str (text), bool (True or False). Check with type().
Type casting	input() always returns text. Use int(), float(), str(), bool() to convert.
Conditions	if, elif, else. Only the first True branch runs. Colon and indentation are required.
Operators	== != > < >= <= for comparing; and, or, not for combining.
for loop	Repeats over a sequence. range(1, 6) gives 1 to 5 (stop is excluded).
while loop	Repeats while a condition is True. Always update something inside it.
break / continue	break stops the loop. continue skips to the next round.

Part B: Going beyond today

1. f-strings: cleaner output

Instead of joining text with commas or plus signs, put an `f` before the quotes and place variables in curly braces.

```
name, score = "Riya", 91.456
print(f"{name} scored {score:.1f}%")   # Riya scored 91.5%
```

2. Integer division and remainder

`//` divides and drops the decimal part. `%` gives the remainder. The remainder is how you check even or odd numbers.

```
print(17 // 5)   # 3
print(17 % 5)    # 2
print(10 % 2 == 0)  # True, so 10 is even
```

3. Why 0.1 + 0.2 is not exactly 0.3

Computers store decimals in binary, so some values are tiny approximations. For money, round your results or, in real projects, use the `decimal` module.

```
print(0.1 + 0.2)            # 0.30000000000000004
print(round(0.1 + 0.2, 2))  # 0.3
```

4. Truthy and falsy values

Python treats some values as False in a condition: `0`, `0.0`, an empty string `""` and `None`. Almost everything else counts as True.

```
name = input("Name: ")
if not name:
    print("You did not type anything")
```

5. Safe type casting with try and except

If a user types "abc" where a number is expected, `int()` crashes with a ValueError. You can catch that and ask again. We cover exceptions fully on Day 3; this is a sneak preview.

```
while True:
    try:
        age = int(input("Age: "))
        break
    except ValueError:
        print("Please type digits only")
```

6. The match statement (Python 3.10 and later)

When you compare one value against many fixed options, `match` can be easier to read than a long elif chain.

```
choice = "2"
match choice:
    case "1": print("Start")
    case "2": print("Settings")
    case _:   print("Unknown option")
```

7. Style matters: PEP 8

PEP 8 is Python's official style guide. The short version: 4 spaces per indent, lowercase_with_underscores for variable names, spaces around operators, and lines that are easy to read. Clean code is easier for your teammates and your future self.

Part C: Check yourself

- What is the difference between = and ==?
- Why does `"5" + "5"` give "55" and not 10?
- When would you choose a while loop over a for loop?

- What does `range(2, 10, 2)` produce?
- What happens if a while loop's condition never becomes False?

Part D: Practice ideas for tonight

- Write a tip calculator: ask for the bill and tip percentage, then print the tip and total.
- Print every number from 1 to 30, but print "Fizz" for multiples of 3 and "Buzz" for multiples of 5.
- Ask the user for a password until they type the correct one, with a maximum of 3 attempts.

Coming up on Day 2: functions (reusable blocks of code), lists, tuples, sets and dictionaries, and reading and writing CSV and JSON files.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today